TalentHub

منصة متكاملة لإدارة التوظيف والمهارات

تقرير مشروع التخرج

وصف كامل للمشروع – معماريته – تقنياته – ميزاته

المشروع **TalentHub**	نوع المشروع **مشروع تخرج**
التقنية الأساسية **Node.js + Express**	قاعدة البيانات **SQLite / PostgreSQL**
اللغة الأمامية **HTML5 + Bootstrap 5**	الاختبارات **Vitest + Playwright**

فهرس المحتويات

1 نظرة عامة على المشروع ورؤيته

2 المعمارية التقنية والتقنيات المستخدمة

3 هيكل الملفات والمجلدات

4 نموذج قاعدة البيانات والكيانات

5 واجهة برمجة التطبيقات (نقاط النهاية)

6 أدوار المستخدمين وصلاحياتهم

7 الواجهة الأمامية ولوحات التحكم

8 نظام المصادقة والأمان

9 الاختبارات وضمان الجودة

10 كيفية تشغيل المشروع

11 النشر على السحابة

12 الخلاصة ونقاط القوة

١. نظرة عامة على المشروع

TalentHub – تقرير مشروع التخرج

ما هي منصة تالنت هب؟

منصة **تالنت هب** هي نظام متكامل لإدارة التوظيف والمهارات، مصمّم ليربط بين ثلاثة أطراف رئيسية: **الطلاب** الباحثين عن فرص عمل، و**الشركات** المُعلِنة عن وظائف، و**الجامعات** الراعية لطلابها – وكل ذلك تحت إشراف **مسؤول النظام**.

تدعم المنصة اللغة العربية بالكامل كلغة أساسية مع دعم اتجاه النص من اليمين لليسار، كما تتيح التبديل بين الوضع الفاتح والداكن وتدعم اللغة الإنجليزية كخيار ثانوي.

٤	٨	+٢٥	٦
أدوار مستخدمين	كيانات في قاعدة البيانات	نقطة نهاية API	اختبارات آلية تنجح

الهدف من المشروع

- ✅ توفير منصة رقمية مركزية تجمع سوق العمل والتعليم الجامعي
- ✅ تمكين الطالب من عرض مهاراته والتقدم للوظائف مباشرة
- ✅ تمكين الشركات من نشر فرص العمل وتصفية المتقدمين
- ✅ تمكين الجامعات من متابعة الطلاب المنتسبين إليها
- ✅ إتاحة لوحة إدارة شاملة للمسؤول للإشراف على النظام بالكامل

أبرز الميزات

🔐 المصادقة الأمنية

نظام تسجيل دخول بـ JWT مع رموز وصول (١٥ دقيقة) ورموز تجديد (٧ أيام)، وتشفير كلمات المرور بـ bcrypt

📊 لوحات التحكم المتعددة

لكل دور لوحة تحكم مخصصة بصلاحيات محددة، تُعرض بعد تسجيل الدخول تلقائياً

🌐 واجهة عربية كاملة

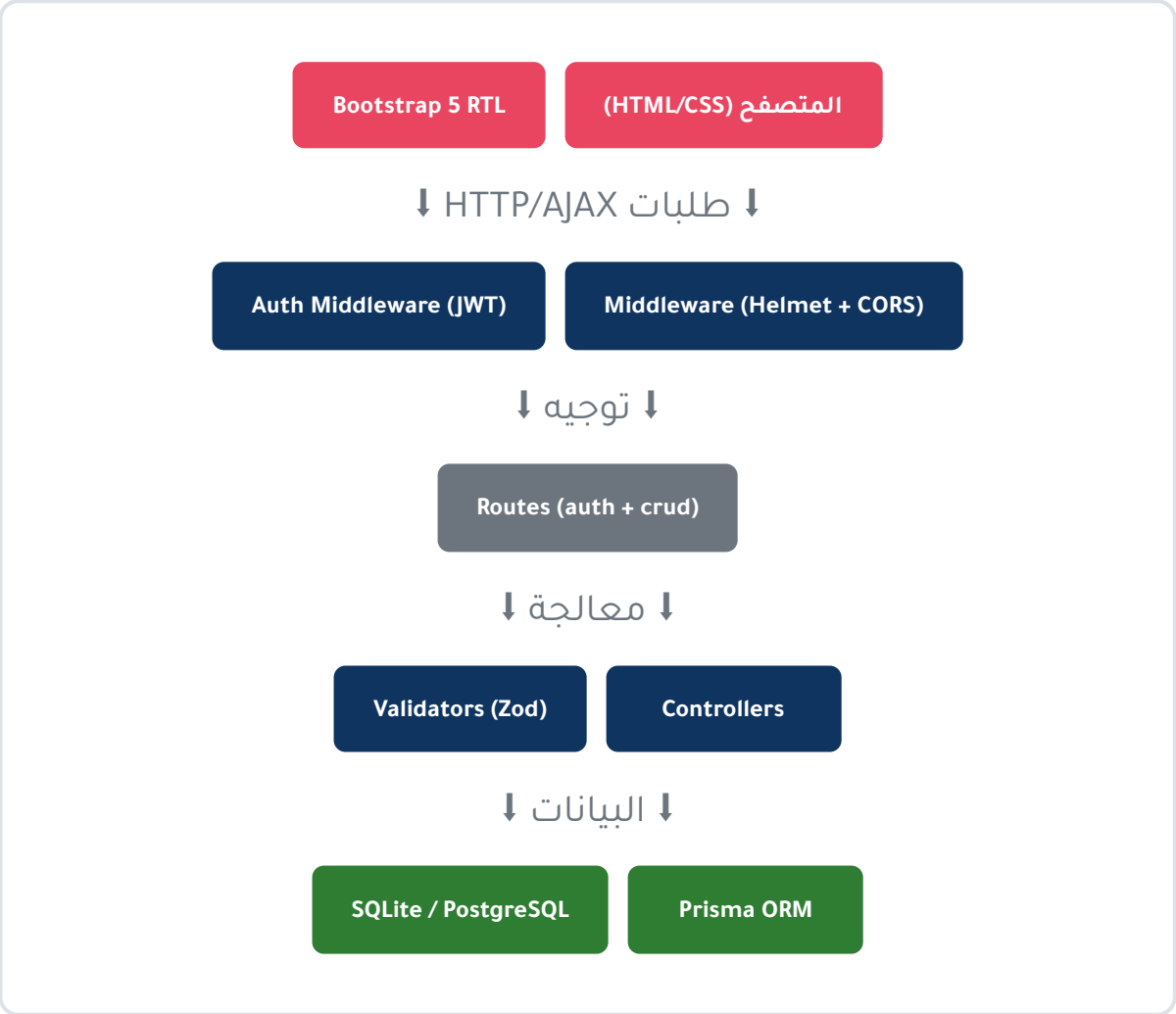
دعم RTL كامل مع Bootstrap 5، تبديل اللغة، وتبديل الوضع الفاتح/الداكن

✅ جودة الكود

ESLint + Vitest + Playwright + GitHub Actions CI = صفر ثغرات أمنية

نمط المعمارية

يعتمد المشروع نمط **MVC (نموذج - عرض - متحكم)** في الطبقة الخلفية، مع فصل واضح بين طبقات التطبيق:



التقنيات المستخدمة

الطبقة	التقنية	الاستخدام	النسخة
الخادم الخلفي	`Node.js`	بيئة تشغيل JavaScript على الخادم	+٢٠
الخادم الخلفي	`Express.js`	إطار عمل HTTP API	x.٤

الطبقة	التقنية	الاستخدام	النسخة
قاعدة البيانات	`Prisma ORM`	طبقة الوصول للبيانات + Migrations	x.٦
قاعدة البيانات	`SQLite`	قاعدة البيانات في التطوير	–
قاعدة البيانات	`PostgreSQL`	قاعدة البيانات في الإنتاج (السحابة)	–
المصادقة	`JWT`	رموز الوصول والتجديد	جوزة ويب JSON
المصادقة	`bcrypt`	تشفير كلمات المرور	–
التحقق	`Zod`	التحقق من مدخلات API	x.٣
الأمان	`Helmet.js`	ترويسات HTTP الأمنية	–
التسجيل	`Pino`	تسجيل السجلات (Logging)	–
الواجهة الأمامية	`Bootstrap 5 RTL`	تصميم متجاوب عربي	x.٥
الاختبار الوحدوي	`Vitest + Supertest`	اختبارات الوحدة والتكامل	–
اختبار المتصفح	`Playwright`	اختبارات E2E شاملة	–
جودة الكود	`ESLint 9`	فحص جودة الكود تلقائياً	x.٩
التكامل المستمر	`GitHub Actions`	CI/CD تلقائي عند كل رفع	–
النشر	`Render.com`	نشر سحابي مجاني	–

٣. هيكل الملفات والمجلدات

هيكل الجذر

```
مشروع التخرج/
├── backend/            ← الخادم الخلفي (Node.js + Express)
├── dashbord/           ← لوحات التحكم (الواجهة الأمامية)
├── login/              ← صفحة تسجيل الدخول
├── register/           ← صفحة التسجيل
├── dashboard/          ← لوحات تحكم إضافية
├── php-api/            ← بديل PHP (للتجارب المحلية فقط)
├── index.html          ← الصفحة الرئيسية (Landing Page)
├── style.css           ← أنماط الصفحة الرئيسية
├── ui-controls.js      ← لوحة الإعدادات العائمة
├── run-project.bat     ← ملف التشغيل السريع (Windows)
└── render.yaml         ← إعداد النشر السحابي
```

هيكل الخادم الخلفي `backend/`

```
backend/
├── prisma/
│   └── schema.prisma       ← تعريف نماذج قاعدة البيانات
├── src/
│   ├── app.js              ← إعداد Express (Middleware + Routes)
│   ├── server.js           ← نقطة الدخول (تشغيل الخادم)
│   ├── seed.js             ← زرع البيانات الابتدائية
│   ├── utils.js            ← أدوات مساعدة مشتركة
│   ├── config/
│   │   ├── env.js          ← قراءة متغيرات البيئة
│   │   ├── logger.js       ← إعداد Pino Logger
│   │   └── prisma.js       ← عميل قاعدة البيانات
│   ├── controllers/
│   │   ├── auth.controller.js  ← منطق المصادقة
│   │   └── crud.controller.js  ← منطق CRUD العام
│   ├── middlewares/
│   │   ├── auth.js         ← التحقق من JWT + الأذونات
│   │   ├── error-handler.js← معالجة الأخطاء
│   │   └── validate.js     ← التحقق من المدخلات (Zod)
│   ├── routes/
│   │   ├── auth.routes.js  ← مسارات المصادقة
│   │   └── crud.routes.js  ← مسارات CRUD لكل الكيانات
│   └── validators/
│       ├── auth.schemas.js ← مخططات التحقق للمصادقة
│       └── resource.schemas.js ← مخططات التحقق للموارد
└── tests/
```

```
    ├── unit/utils.test.js          ← اختبارات الوحدة
    ├── integration/health-auth.test.js ← اختبارات التكامل
    └── e2e/
        ├── landing.spec.js         ← اختبار الصفحة الرئيسية
        └── login.spec.js           ← اختبار تسجيل الدخول
```

هيكل الواجهة الأمامية `dashbord/`

```
dashbord/
├── api.js                  ← وطبقة التواصل API تكوين
├── dashbord.html/css       ← لوحة التحكم الرئيسية
├── admin/                  ← لوحة المسؤول
│   ├── dashbord.html       ← الرئيسية الإدارية
│   ├── admin-students.html ← إدارة الطلاب
│   ├── admin-companies.html← إدارة الشركات
│   ├── admin-universities.html ← إدارة الجامعات
│   ├── admin-jobs.html     ← إدارة الوظائف
│   └── admin-skills.html   ← إدارة المهارات
├── student/                ← لوحة الطالب
│   ├── dashboard.html      ← رئيسية الطالب
│   ├── jobs.html           ← استعراض الوظائف
│   ├── my-applications.html← طلباتي
│   └── my-profile.html     ← ملفي الشخصي
├── company/                ← لوحة الشركة
│   ├── dashboard.html
│   ├── my-jobs.html
│   ├── post-job.html
│   └── applications.html
└── university/             ← لوحة الجامعة
    ├── dashboard.html
    ├── students.html
    ├── departments.html
    └── partnerships.html
```

٤. نموذج قاعدة البيانات

يعتمد المشروع **Prisma ORM** مع قاعدة بيانات **SQLite** في بيئة التطوير و**PostgreSQL** في الإنتاج. فيما يلي الكيانات الثمانية وعلاقاتها:

الكيان	الحقول الرئيسية	العلاقات
User المستخدم	id, fullName, email, passwordHash, role, status, refreshToken	يرتبط بـ Student أو Company أو University (واحد لكل)
Student الطالب	id, userId, universityId, bio, status	يتبع User، ينتمي لـ University، يملك مهارات وطلبات توظيف
Company الشركة	id, userId, name, industry, status	يتبع User، يملك وظائف متعددة
University الجامعة	id, userId, name, location, status	يتبع User، يضم طلاباً متعددين
Job الوظيفة	id, companyId, title, location, type, status, description	تتبع Company، تحتاج مهارات، تستقبل طلبات
Skill المهارة	id, name, category, status	تُربط بالطلاب والوظائف (Many-to-Many)
StudentSkill مهارات الطالب	studentId, skillId	جدول وسيط (Many-to-Many) بين Student و Skill
Application طلب التوظيف	id, studentId, jobId, status	يربط Student بـ Job، يمنع التكرار (@@unique)

علاقات الكيانات (مبسّطة)

قيم الحالة (Status)

الكيان	القيم المتاحة	المعنى
User	`ACTIVE` / `INACTIVE`	حالة الحساب
Student / Company / University	`PENDING` / `APPROVED` / `REJECTED`	حالة الموافقة من المسؤول
Job	`OPEN` / `CLOSED`	حالة إتاحة الوظيفة
Application	`PENDING` / `ACCEPTED` / `REJECTED`	مراجعة طلب التوظيف

٥. واجهة برمجة التطبيقات (API)

عنوان القاعدة: `http://localhost:4000/api` | البروتوكول: HTTP REST | صيغة البيانات: JSON

نقاط نهاية المصادقة

الطريقة	المسار	الوصف	المصادقة المطلوبة
POST	`/auth/register`	تسجيل مستخدم جديد	لا
POST	`/auth/login`	تسجيل الدخول والحصول على رموز	لا
POST	`/auth/refresh`	تجديد رمز الوصول	لا (refreshToken)
GET	`/auth/me`	بيانات المستخدم الحالي	نعم
POST	`/auth/logout`	تسجيل الخروج	نعم

نقاط نهاية CRUD (لكل كيان)

الطريقة	المسار	الوصف	الأدوار المسموحة
GET	`/users`	قائمة المستخدمين	ADMIN
GET	`/students`	قائمة الطلاب	ADMIN, UNIVERSITY
GET	`/companies`	قائمة الشركات	ADMIN
GET	`/universities`	قائمة الجامعات	ADMIN
GET	`/jobs`	قائمة الوظائف	ADMIN, STUDENT, COMPANY
GET	`/skills`	قائمة المهارات	ADMIN, STUDENT
GET	`/applications`	قائمة طلبات التوظيف	ADMIN, STUDENT, COMPANY
لكل مسار أعلاه:	`GET /:id` | `POST /` | `PUT /:id` | `DELETE /:id`		

نقطة نهاية الفحص الصحي

المسار	الاستجابة	الاستخدام
`GET /api/health`	`{"status":"ok","timestamp":"..."}`	التحقق من أن الخادم يعمل

مثال على استجابة API ناجحة

```
// POST /api/auth/login
// الطلب:
{
  "email": "admin@talenthub.local",
  "password": "Admin1234"
}

// الاستجابة (200):
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "clxxx...",
    "fullName": "System Administrator",
    "email": "admin@talenthub.local",
    "role": "ADMIN"
  }
}
```

٦. أدوار المستخدمين وصلاحياتهم

يدعم النظام أربعة أدوار متمايزة، لكل منها صلاحيات وواجهة خاصة:

👑 المسؤول (ADMIN)

- عرض وإدارة جميع المستخدمين
- الموافقة على تسجيل الطلاب والشركات والجامعات
- إدارة المهارات وتصنيفها
- عرض جميع الوظائف وطلبات التوظيف
- الوصول لجميع نقاط النهاية
- التوجيه إلى:

`/dashbord/admin/dashbord.html`

🎓 الطالب (STUDENT)

- عرض الوظائف المتاحة
- التقدم للوظائف
- إدارة ملفه الشخصي والمهارات
- متابعة حالة طلباته
- التوجيه إلى:

`/dashbord/student/dashboard.html`

🏢 الشركة (COMPANY)

- نشر وظائف جديدة
- إدارة الوظائف المنشورة
- مراجعة طلبات الطلاب
- قبول أو رفض المتقدمين
- التوجيه إلى:

`/dashbord/company/dashboard.html`

🏛 الجامعة (UNIVERSITY)

- عرض الطلاب المنتسبين
- متابعة نشاط الطلاب
- إدارة بيانات الجامعة والأقسام
- عرض الشراكات مع الشركات
- التوجيه إلى:

`/dashbord/university/dashboard.html`

آلية التوجيه بعد تسجيل الدخول

```
// التوجيه التلقائي حسب الدور — dashbord/api.js في
const redirectMap = {
  ADMIN:      "/dashbord/admin/dashbord.html",
  STUDENT:    "/dashbord/student/dashboard.html",
  COMPANY:    "/dashbord/company/dashboard.html",
  UNIVERSITY: "/dashbord/university/dashboard.html"
};
window.location.href = redirectMap[user.role];
```

حماية المسارات في الخادم

```
// middleware/auth.js — التحقق من الدور
function requireRole(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: "Forbidden" });
    }
    next();
  };
}
```

٧. الواجهة الأمامية ولوحات التحكم

صفحات النظام

الصفحة	المسار	الوصف	الدور
الصفحة الرئيسية	`/index.html`	Landing Page للمنصة	الجميع
تسجيل الدخول	`/login/login.html`	نموذج الدخول	الجميع
التسجيل	`/register/register.html`	إنشاء حساب جديد	الجميع
لوحة المسؤول	`/dashbord/admin/dashbord.html`	الرئيسية مع إحصائيات	ADMIN
إدارة الطلاب	`/dashbord/admin/admin-students.html`	CRUD للطلاب	ADMIN
إدارة الشركات	`/dashbord/admin/admin-companies.html`	CRUD للشركات	ADMIN
إدارة الجامعات	`/dashbord/admin/admin-universities.html`	CRUD للجامعات	ADMIN
إدارة الوظائف	`/dashbord/admin/admin-jobs.html`	CRUD للوظائف	ADMIN
إدارة المهارات	`/dashbord/admin/admin-skills.html`	CRUD للمهارات	ADMIN
وظائف الطالب	`/dashbord/student/jobs.html`	استعراض والتقدم	STUDENT
طلبات الطالب	`/dashbord/student/my-applications.html`	متابعة الطلبات	STUDENT
ملف الطالب	`/dashbord/student/my-profile.html`	تعديل البيانات والمهارات	STUDENT

الصفحة	المسار	الوصف	الدور
نشر وظيفة	`/dashbord/company/post-job.html`	إضافة وظيفة جديدة	COMPANY
طلاب الجامعة	`/dashbord/university/students.html`	قائمة الطلاب	UNIVERSITY

ميزات الواجهة

🌙 **الوضع الداكن/الفاتح**
تبديل فوري محفوظ في LocalStorage

🌐 **ثنائية اللغة**
عربي (RTL) وإنجليزي – تحويل آني

📱 **تصميم متجاوب**
Bootstrap 5 – يعمل على الجوال والكمبيوتر

آلية اكتشاف API `(dashbord/api.js)`

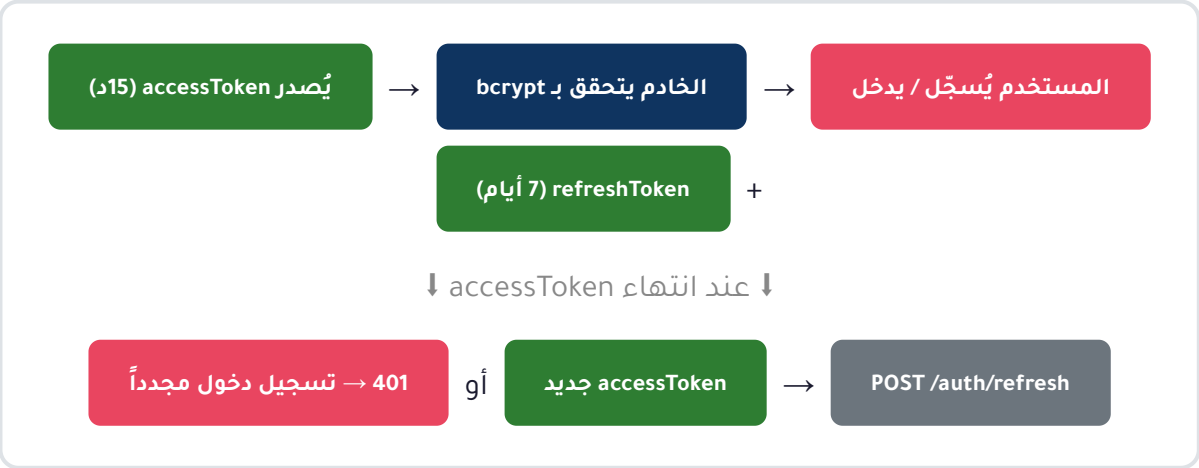
يستخدم النظام آلية fallback ذكية لاكتشاف عنوان API تلقائياً:

```
async function resolveApiBase() {
  // جرّب أولاً: Node.js backend
  try {
    await fetch("http://localhost:4000/api/health");
    return "http://localhost:4000/api";
  } catch {}
  // جرّب ثانياً: PHP backend
  return "/php-api/index.php";
}
```

طبقات الأمان المُطبَّقة

الطبقة	الأداة	الحماية التي توفرها
ترويسات HTTP	Helmet.js	XSS، Clickjacking، MIME sniffing، HSTS
سياسة الأصول	CORS مُهيَّأ	يقبل فقط النطاقات المُصرَّح بها
كلمات المرور	bcrypt (rounds=10)	لا تُخزَّن كلمات المرور نصياً أبداً
رمز الوصول	JWT (HS256)	صلاحية ١٥ دقيقة، يُرسَل في Authorization Header
رمز التجديد	JWT (HS256)	صلاحية ٧ أيام، يُخزَّن مُشفَّراً في قاعدة البيانات
التحقق من المدخلات	Zod	يرفض أي مدخل غير متوقع قبل وصوله للقاعدة
التحقق من الدور	requireRole middleware	يمنع الوصول غير المصرح للموارد
ثغرات الحزم	npm audit	صفر ثغرات (تم تحديث Vite لإصلاح ثغرة path traversal)

دورة حياة المصادقة



مثبتات الأمان في ملف البيئة `.env`

```
NODE_ENV=development
PORT=4000
DATABASE_URL="file:../prisma/dev.db"
JWT_SECRET=your-super-secret-key-min-32-chars
JWT_REFRESH_SECRET=your-refresh-secret-key-min-32-chars
JWT_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
CORS_ORIGIN=http://127.0.0.1:5500
```

✅ **نتيجة الفحص الأمني:** تم تشغيل `npm audit` وتم تصحيح جميع الثغرات – النتيجة: **صفر ثغرات (0 vulnerabilities)**. تم تحديث Vite من 8.0.0 إلى 8.0.8 لإصلاح ثغرة path traversal الخطيرة.

٩. الاختبارات وضمان الجودة

6	6/6	0	✓
مجموع الاختبارات	اختبارات تنجح	ثغرات أمنية	CI يعمل

أنواع الاختبارات

النوع	الأداة	الملف	ما يختبره	النتيجة
وحدة	Vitest	`tests/unit/utils.test.js`	دوال الأدوات المساعدة	✓ نجاح
تكامل	Vitest + Supertest	`tests/integration/health-auth.test.js`	Health endpoint + تسجيل + دخول	✓ نجاح
E2E	Playwright	`tests/e2e/landing.spec.js`	الصفحة الرئيسية – ظهور زر CTA	✓ نجاح
E2E	Playwright	`tests/e2e/login.spec.js`	تسجيل الدخول + التوجيه حسب الدور	✓ نجاح

أوامر الاختبار

```
# دفعة واحدة Lint + تشغيل جميع الاختبارات
npm run check:all

# اختبارات الخادم الخلفي فقط (Vitest)
npm run test:api

# اختبارات المتصفح (Playwright E2E)
npm run test:e2e

# فحص جودة الكود (ESLint)
npm run lint:api
```

التكامل المستمر (GitHub Actions)

يوجد ملف `.github/workflows/ci-check.yml` يُنفَّذ تلقائياً عند كل رفع (push) أو طلب دمج (pull request) على فروع `master` و `main` :

```
# خطوات CI تلقائياً:
1. تثبيت Node.js 20
2. تثبيت حزم npm
3. توليد Prisma Client
4. تطبيق مخطط قاعدة البيانات
5. تثبيت متصفح Playwright (Chromium)
6. تشغيل: npm run check:all
   → ESLint ✓
   → Vitest (unit + integration) ✓
   → Playwright E2E ✓
```

نمط اختبار E2E – تسجيل الدخول مع محاكاة الشبكة

```
// tests/e2e/login.spec.js
test("successful login redirects admin to dashboard", async ({ page }) => {
  // بدون خادم حقيقي API محاكاة استجابة
  await page.route("**/auth/login", route => route.fulfill({
    json: { accessToken: "tok", user: { role: "ADMIN" } }
  }));

  await page.fill("#email", "admin@talenthub.local");
  await page.fill("#password", "Admin1234");
  await page.click('button[type="submit"]');

  // التحقق من التوجيه الصحيح
  await expect(page).toHaveURL(/dashbord\/admin\/dashbord\.html/);
});
```

١٠. كيفية تشغيل المشروع

الطريقة الأولى: تشغيل سريع بضغطة واحدة (Windows)

الأسهل والأسرع: انقر نقراً مزدوجاً على ملف `run-project.bat` في جذر المشروع.

سيقوم تلقائياً بتشغيل الخادم الخلفي + خادم الواجهة + فتح المتصفح على صفحة تسجيل الدخول.

الطريقة الثانية: التشغيل اليدوي خطوة بخطوة

الخطوة ١ – تثبيت الحزم (مرة واحدة)

```
npm --prefix backend install
```

الخطوة ٢ – إعداد قاعدة البيانات (مرة واحدة)

```
# نسخ ملف البيئة
copy backend\.env.example backend\.env

# توليد Prisma Client
npm --prefix backend run prisma:generate

# تطبيق مخطط قاعدة البيانات
npm --prefix backend run prisma:push

# زرع بيانات المسؤول الابتدائية
npm run seed:api
```

الخطوة ٣ – تشغيل الخادم الخلفي

```
# في نافذة أوامر أولى
npm run dev:api
# الخادم يبدأ على: http://localhost:4000
```

الخطوة ٤ – تشغيل الواجهة الأمامية

```
# في نافذة أوامر ثانية
npx http-server -p 5500 -c-1
# الواجهة تبدأ على: http://127.0.0.1:5500
```

الخطوة ٥ – فتح المتصفح

```
http://127.0.0.1:5500/login/login.html
```

بيانات الحسابات التجريبية

الدور	البريد الإلكتروني	كلمة المرور	الواجهة التي تفتح
مسؤول	`admin@talenthub.local`	`Admin1234`	لوحة المسؤول الكاملة

لإضافة طلاب وشركات وجامعات تجريبية: `npm --prefix backend run seed`

المتطلبات التقنية

المتطلب	الإصدار الأدنى	كيف تتحقق
Node.js	20.0.0	`node --version`
npm	10.0.0	`npm --version`
متصفح حديث	Chrome / Edge / Firefox	–

١١. النشر على السحابة

النشر عبر Render.com

يحتوي المشروع على ملف `render.yaml` الجاهز للنشر التلقائي على منصة Render.com (مجانية للمشاريع الصغيرة).

خطوات النشر

1. أنشئ حساباً مجانياً على `render.com`
2. اربط الحساب بمستودع GitHub
3. اختر "New Web Service" وحدد المستودع
4. Render سيقرأ `render.yaml` تلقائياً ويُهيّئ الخدمة
5. أضف متغيرات البيئة في إعدادات الخدمة (DATABASE_URL، JWT_SECRET، إلخ)

ملف `render.yaml`

```
services:
  - type: web
    name: talenthub-api
    env: node
    buildCommand: npm install && npx prisma generate && npx prisma migrate deplo
    startCommand: node src/server.js
    envVars:
      - key: NODE_ENV
        value: production
      - key: DATABASE_URL
        sync: false    # تُدخَل يدوياً في لوحة Render
      - key: JWT_SECRET
        sync: false
```

إعدادات الإنتاج المطلوبة

المتغير	القيمة المطلوبة في الإنتاج
`DATABASE_URL`	رابط PostgreSQL من Render أو Supabase
`JWT_SECRET`	نص عشوائي طويل (32+ حرف)

المتغير	القيمة المطلوبة في الإنتاج
`JWT_REFRESH_SECRET`	نص عشوائي مختلف (32+ حرف)
`CORS_ORIGIN`	رابط الواجهة الأمامية المنشورة
`NODE_ENV`	`production`

ملخص المشروع

منصة **تالنت هب** هي مشروع تخرج متكامل يُجسّد الجانب العملي لتطوير تطبيقات الويب الحديثة. يجمع المشروع بين الواجهة الأمامية العربية الكاملة والخادم الخلفي المبني بأفضل الممارسات، مع اهتمام واضح بالجودة والأمان والاختبار الآلي.

نقاط القوة الرئيسية

#	نقطة القوة	الدليل
١	معمارية نظيفة ومنظّمة	فصل واضح: routes → controllers → validators → prisma
٢	أمان متعدد الطبقات	Helmet + CORS + JWT + bcrypt + Zod = حماية شاملة
٣	نظام مصادقة احترافي	Access Token (15د) + Refresh Token (7 أيام) + تخزين آمن
٤	اختبارات آلية شاملة	٦ اختبارات: وحدة + تكامل + E2E كلها تنجح
٥	صفر ثغرات أمنية	npm audit → 0 vulnerabilities بعد تحديث Vite
٦	CI/CD تلقائي	GitHub Actions يُشغّل كل الاختبارات عند كل push
٧	دعم لغوي كامل	عربي RTL + إنجليزي + وضع داكن + متجاوب
٨	جاهز للنشر السحابي	render.yaml + .env.production.example جاهزان
٩	توثيق شامل	SETUP_INSTRUCTIONS + README + وصايا.md + هذا التقرير
١٠	سهولة التشغيل	run-project.bat – ضغطة واحدة تشغّل كل شيء

النتيجة النهائية:

`npm run check:all` → Vitest ✅ | ESLint (4 اختبارات) ✅ | Playwright E2E (2 اختبارات) ✅

`npm audit` → **0 vulnerabilities** ✅

`GET /api/health` → `{"status":"ok"}` ✅

TalentHub

منصة متكاملة لإدارة التوظيف والمهارات

مشروع التخرج – تقرير شامل